

Modbus项目开发学习笔记

目录

Modbus项目开发学习笔记

目录

一、硬件

二、CubeMX生成

1. 选择芯片
2. 配置调试接口
3. 配置HAL时基
4. 配置外部高速时钟
5. 配置系统时钟
6. 配置GPIO
7. 配置USART1
8. 生成IAR

三、HAL库接口

1. HAL返回状态
2. HAL初始化和系统时间

`HAL_Init()`

`HAL_GetTick()`、`HAL_IncTick()` 和 `HAL_Delay()`

3. UART初始化结构体
4. `HAL_UART_Init()`
5. `HAL_UART_Transmit()`
6. `HAL_UART_Receive_IT()`
7. UART中断和回调
8. GPIO结构体和函数
9. 时钟结构体和函数
10. NVIC接口
11. HAL接口使用顺序

四、数据与地址分配

1. 线圈区
2. 保持寄存器区
3. HMI变量绑定
4. 功能模块

六、程序启动流程

七、RTU接收与分帧

八、从站协议处理

九、四个功能码

1. `0x01` 读取线圈
2. `0x03` 读取保持寄存器
3. `0x0F` 写多个线圈
4. `0x10` 写多个保持寄存器

十、HMI通信

1. TouchWin函数功能块
2. 公共函数和功能函数
3. TouchWin预定义数据类型
4. HMI内部对象
5. 宏和通信函数

`HIBYTE()` 和 `LOBYTE()`

`Enter()` 和 `Leave()`

`Send()`

`Receive()`

SetPSB() 和 ResetPSB()

6. 四个功能块

读取函数

写入函数

7. 结果码

8. 自定义报文函数块

十、在线状态与运行时间

十一、注意事项

一、硬件

使用TG765-MT触摸屏作为Modbus RTU主站，XDM-60T4-E作为从站设备。XDM内部控制器为STM32F407IG6，程序运行在该芯片内部。触摸屏通过RS-485链路发送Modbus RTU请求，STM32从站接收请求后完成线圈和保持寄存器的读写，并返回应答。

硬件	作用
XDM-60T4-E	运行STM32程序、uC/OS-II和Modbus从站
TG765-MT	作为Modbus主站，发起读写请求并显示结果
ST-Link V2	下载和在线调试STM32程序
XVP通信线	触摸屏下载及设备通信
USB-COM	串口抓包和辅助验证

ST-Link只负责下载和调试，不参与Modbus数据传输。触摸屏下载线用于下载画面程序，也不能替代PLC口的RS-485通信。真正传输Modbus请求和应答的是触摸屏与XDM之间的RS-485链路。

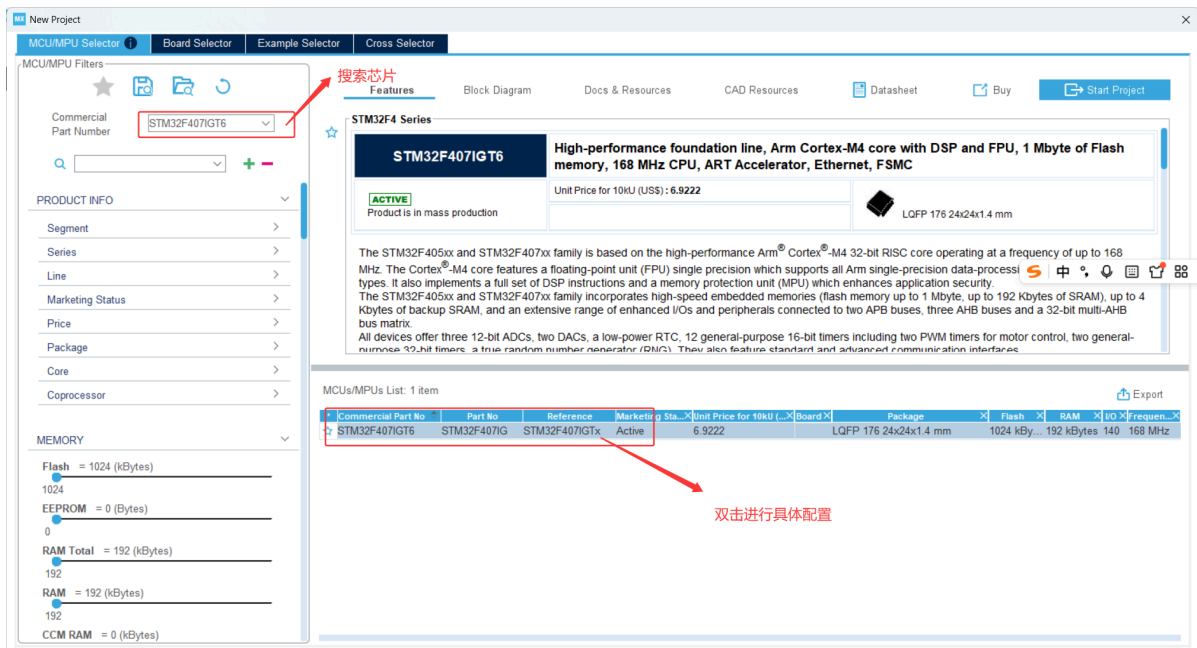
二、CubeMX生成

完成芯片选择、调试接口、时钟、GPIO、USART1和IAR文件生成。

1. 选择芯片

打开STM32CubeMX，新建并进入MCU选择界面。在搜索框输入STM32F407IG，选择STM32F407IGTX。

芯片型号决定启动文件、中断向量表、外设寄存器定义、Flash/RAM容量和引脚资源。型号选择错误时，后续生成的可能出现启动文件不匹配、外设不存在、引脚对不上等问题。



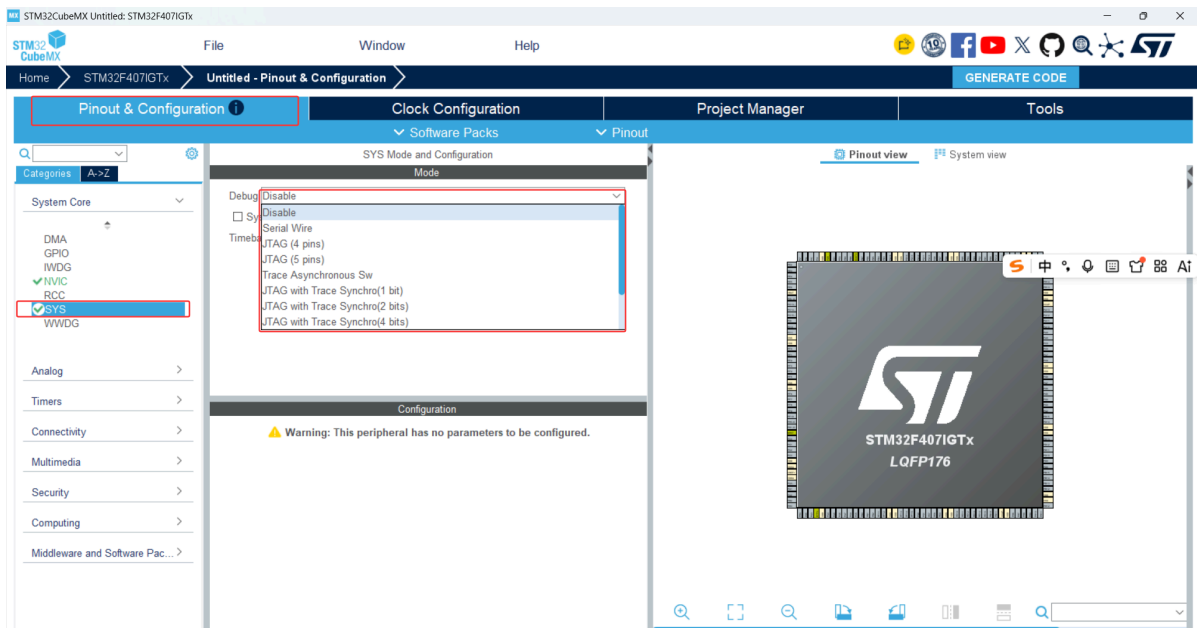
需要确认的内容：

- CubeMX中选择 STM32F407IGTx
- 板卡实际芯片为 STM32F407IG6
- 已安装或下载STM32CubeF4固件包

2. 配置调试接口

进入 Pinout & Configuration，选择 System Core -> SYS，将 Debug 设置为 Serial Wire。

SWD用于ST-Link下载和在线调试，只需要 SWDIO、SWCLK 和 GND。如果没有打开SWD，程序下载后可能把调试引脚当作普通GPIO使用，导致后续无法再次连接芯片。



注意事项：

- 新建后先配置SWD
- 不要把SWD引脚重新配置为普通GPIO
- 无法下载或无法重新连接时，优先检查该配置

3. 配置HAL时基

当前使用 `SystemTick` 作为HAL时基，同时作为uC/OS-II系统节拍。`HAL_Init()` 把SysTick配置为1 ms周期，`SystemTick_Handler()` 在每次中断中依次维护HAL和uC/OS-II。

```
void SystemTick_Handler(void)
{
    HAL_IncTick();
    if (OSRunning == OS_TRUE)
    {
        OS_CPU_SystemTickHandler();
    }
}
```

`HAL_IncTick()` 增加HAL毫秒计数，供 `HAL_GetTick()` 和HAL超时机制使用。

`OS_CPU_SystemTickHandler()` 进入uC/OS-II节拍处理，用于任务延时、事件超时和调度。两套计数来源相同，但保存位置和用途不同。

HAL和uCOS-II可以共用SysTick，前提是中断入口只有一个，并且两个节拍处理函数都被调用。

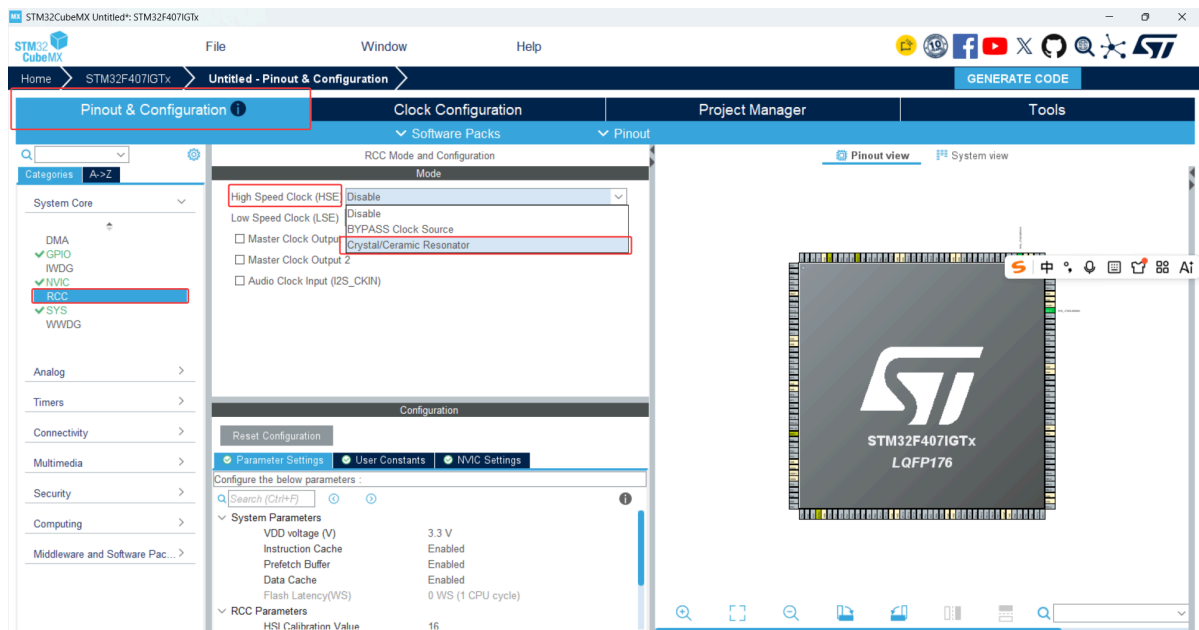
注意事项：

- `HAL_GetTick()` 来自HAL时基，`OSTimeGet()` 来自uC/OS-II节拍
- 修改时基后必须检查中断入口，避免漏调或重复调用 `HAL_IncTick()`
- Modbus RTU的帧间隔不再使用DWT周期计数器，当前由TIM2一次定时器判断t3.5静默时间

4. 配置外部高速时钟

进入 `System Core -> RCC`，将 `High Speed Clock (HSE)` 设置为 `Crystal/Ceramic Resonator`。

XDM板载外部晶振为 12 MHz。外部晶振比内部RC振荡器稳定，适合作为系统时钟和串口波特率的基础。串口通信对时钟误差敏感，时钟不准时容易出现乱码、CRC错误和主站超时。



需要确认的内容：

- `HSE` 选择外部晶振模式

- HSE不是最终系统时钟

5. 配置系统时钟

进入 **Clock Configuration**，选择HSE作为PLL输入，并将系统时钟配置到 **168 MHz**。

STM32 的外设默认时钟是关闭的，用之前必须先开时钟，否则寄存器读写无效。

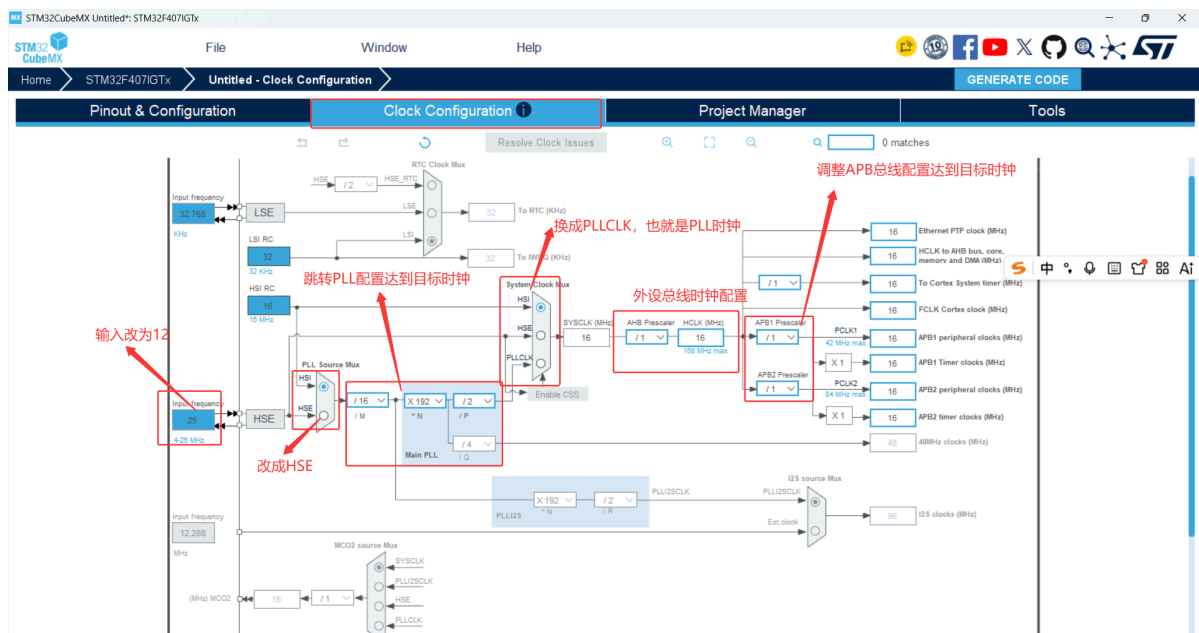
本采用的时钟链路为：

HSE 12 MHz

- > PLLM = 12
- > PLL输入为1 MHz
- > PLLN = 336
- > VCO为336 MHz
- > PLLP = 2
- > SYSCLK为168 MHz
- > AHB不分频，HCLK为168 MHz
- > APB1四分频，PCLK1为42 MHz
- > APB2二分频，PCLK2为84 MHz

STM32F407系统时钟最高为 **168 MHz**。AHB供给内核、存储器和DMA等高速部分。

APB1和APB2供给外设，两条总线有各自的频率上限，不能都直接使用168 MHz。

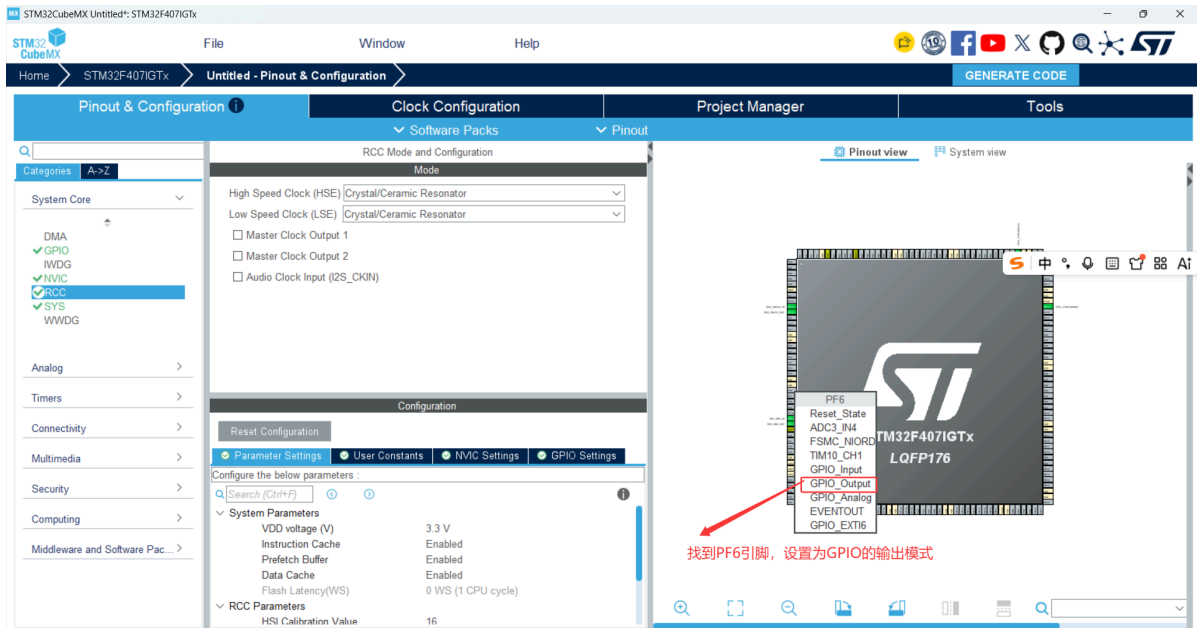


- **System Clock Mux** 选择 **PLLCLK**
- **PLLM=12** 用于把12 MHz降到1 MHz
- **PLLN=336** 用于得到336 MHz VCO
- **PLLP=2** 用于得到168 MHz系统时钟
- **APB1** 保持在42 MHz
- **APB2** 保持在84 MHz
- 时钟错误会影响HAL延时、串口波特率和OS节拍

6. 配置GPIO

在引脚配置界面中，将Q0对应的PF6配置为普通输出。该输出用于基础点灯验证，也可在后续作为运行状态或通信状态指示。

点灯验证不依赖uC/OS-II和Modbus。基础刚生成时，先让一个GPIO按固定周期翻转，可以快速确认程序已经下载、系统时钟已经运行、HAL延时可用。



GPIO常见工作模式如下：

- 输入浮空。引脚内部不上拉也不下拉，电平完全由外部电路决定，适合外部已经有明确上下拉的输入信号。
- 输入上拉或下拉。芯片内部接入上拉或下拉电阻，引脚悬空时仍有确定电平，常用于按键、拨码开关等输入。
- 模拟输入。关闭数字输入输出通道，用于ADC采样或降低未用引脚功耗。
- 推挽输出。引脚可以主动输出高电平和低电平，适合驱动LED指示、芯片使能脚和普通数字控制信号。若控制继电器、电机等负载，通常还需要三极管、MOS管或驱动芯片。
- 开漏输出。引脚只能主动拉低，输出高电平时处于高阻态，需要外部或内部上拉电阻，常用于I2C等多设备共享线路。
- 复用推挽或复用开漏。引脚控制权交给片上外设，例如USART的TX、SPI的SCK、I2C的SCL，不再由普通GPIO输出寄存器直接控制。

GPIO初始化通常通过 `GPIO_InitTypeDef` 结构体完成，主要配置引脚、模式、上下拉和输出速度。

`Speed` 表示输出信号边沿变化速度，也可理解为输出驱动速度。普通指示灯不需要高速翻转，选择 `GPIO_SPEED_FREQ_LOW` 即可，速度过高反而可能增加电磁干扰。

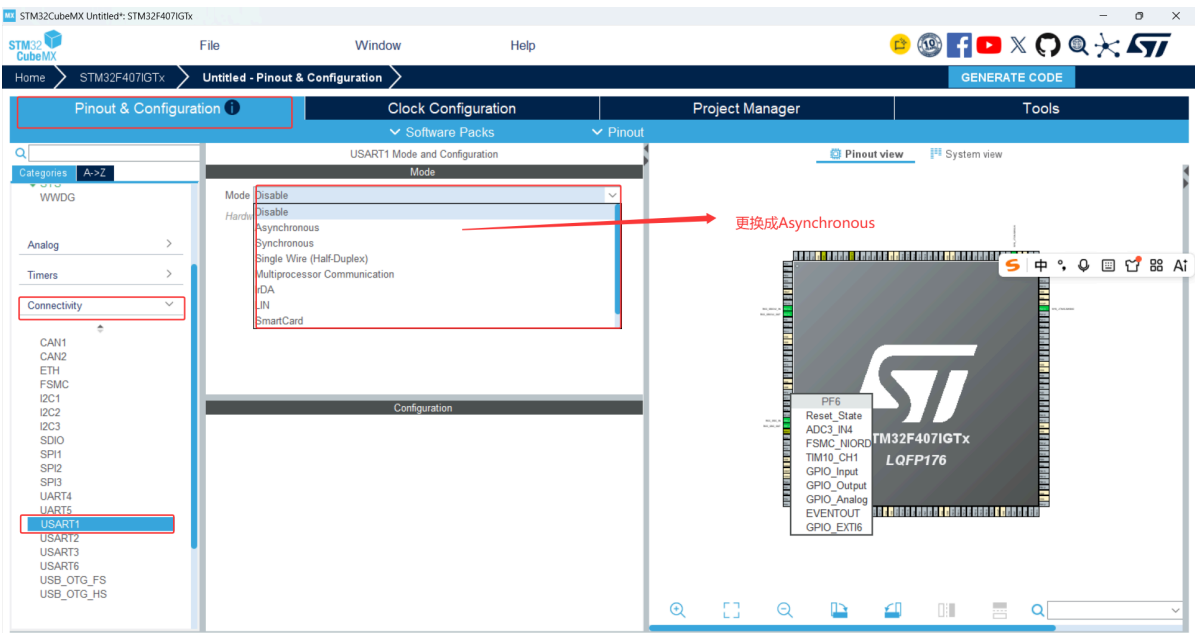
```
GPIO_InitStruct.Pin = Q0_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

注意事项：

- PF6配置为 GPIO_Output
- 输出初始电平结合硬件有效电平确定
- 指示灯GPIO不需要配置为高速输出
- 未确认用途的引脚不要随意配置为输出

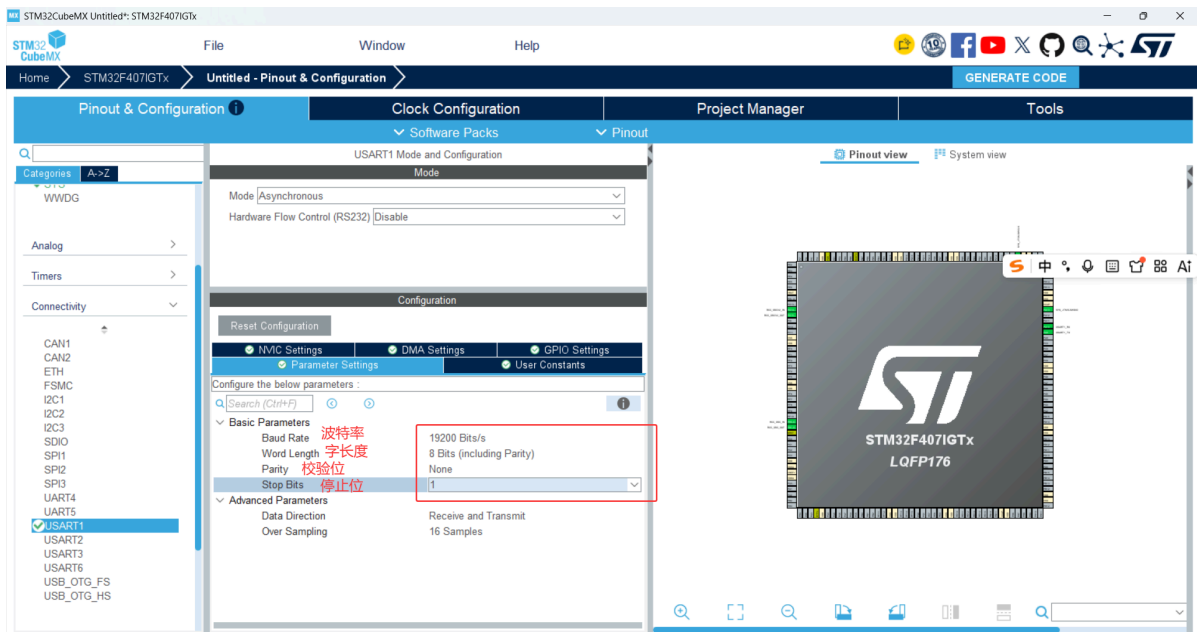
7. 配置USART1

进入 Connectivity -> USART1，模式选择 Asynchronous。PA9作为发送引脚，PA10作为接收引脚。



配置项	数值
Baud Rate	19200
Word Length	8 Bits
Parity	None
Stop Bits	1
Mode	TX and RX

USART1负责STM32侧的字节收发。Modbus RTU只规定报文格式，底层仍然要通过串口把每个字节发出去。主站和从站的波特率、数据位、校验位、停止位必须一致。



实现时需要先指定串口外设，对应配置串口的波特率，数据位长度，停止位，校验位，以及通信模式，如单工、半双工等。

注意事项：

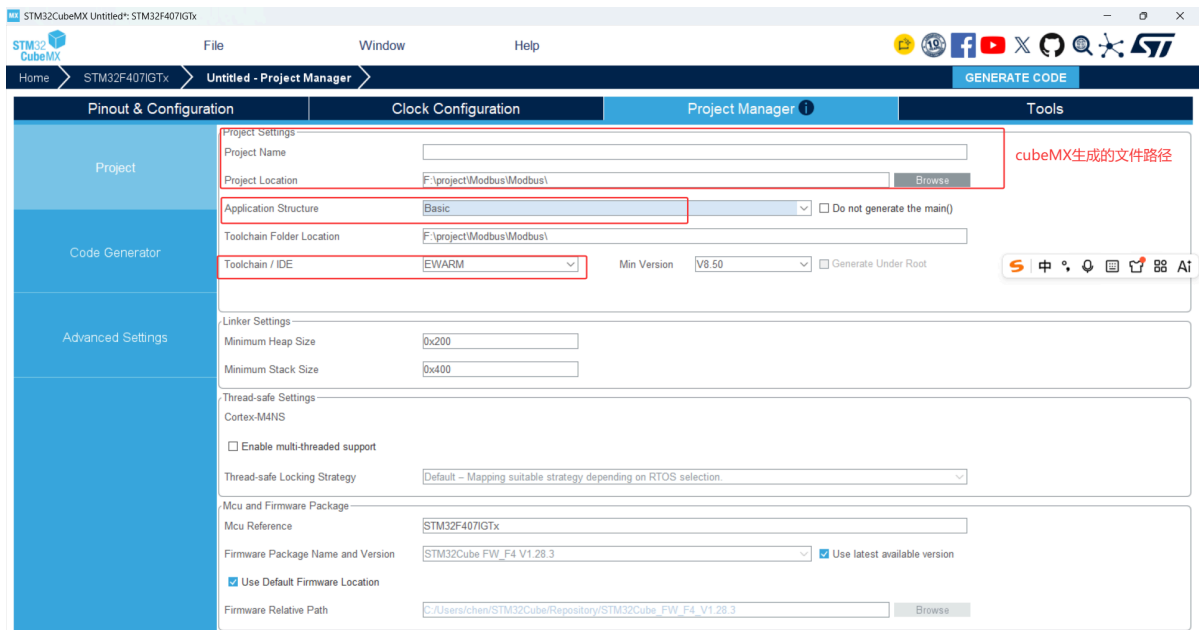
- 本最终联调使用 19200，8-N-1
- N 表示无校验
- 参数不一致时常见现象为CRC错误、接收长度异常或主站超时
- CubeMX只生成串口初始化，RTU分帧仍由应用代码完成
- RS-485方向控制是否需要GPIO，应按XDM硬件电路判断

8. 生成IAR

进入 Project Manager 页面，完成名称、路径和工具链配置。

配置项	设置
Toolchain / IDE	EWARM
Application Structure	Basic
Code Generator	每个外设生成独立 .c/.h 文件
Firmware Package	复制所需库文件到目录

EWARM 会生成IAR可打开的 .eww 和 .ewp 文件。外设初始化拆分为独立文件后，GPIO、USART和时钟配置更容易查找。固件库复制到目录后，迁移和离线编译更稳定。



注意事项：

- 路径尽量使用英文
- 用IAR打开 .eww 工作区文件
- 用户代码写入 USER CODE BEGIN/END 区域
- 重新生成后检查中断文件、初始化函数和分组
- .ioc 是CubeMX配置来源，应随保留

三、HAL库接口

HAL是ST提供的硬件抽象层。应用程序通过统一接口配置和操作外设，HAL再访问STM32寄存器。CubeMX主要生成结构体赋值、初始化函数和中断入口，通信协议、缓冲区管理和异常处理仍由应用程序完成。

1. HAL返回状态

多数可能失败的HAL函数返回 `HAL_StatusTypeDef`。

```
typedef enum
{
    HAL_OK      = 0x00U,
    HAL_ERROR   = 0x01U,
    HAL_BUSY    = 0x02U,
    HAL_TIMEOUT = 0x03U
} HAL_StatusTypeDef;
```

返回值	含义	常见处理
HAL_OK	操作成功	继续执行
HAL_ERROR	参数、配置或硬件状态错误	记录错误，停止当前操作

返回值	含义	常见处理
HAL_BUSY	外设正在执行同类操作	等待本次操作完成，不能覆盖原缓冲区
HAL_TIMEOUT	阻塞操作在规定时间内未完成	放弃本次操作，检查线路和外设状态

返回 HAL_OK 只表示当前HAL调用成功。例如 HAL_UART_Receive_IT() 返回 HAL_OK 表示接收中断已经启动，不表示数据已经收到；真正接收完成后才会进入回调函数。

2. HAL初始化和系统时间

HAL_Init()

```
HAL_StatusTypeDef HAL_Init(void);
```

该函数完成Flash预取和缓存配置、NVIC优先级分组、HAL时基初始化，并调用 HAL_MspInit() 完成底层初始化。成功返回 HAL_OK。它应在复位后的基础硬件状态下调用，并位于系统时钟和外设初始化之前。

HAL_Init() 执行时系统通常仍使用复位后的HSI时钟。SystemClock_Config() 改变系统时钟后，HAL会重新配置SysTick周期，使毫秒时基与新的 SystemCoreClock 一致。

HAL_GetTick()、HAL_IncTick() 和 HAL_Delay()

```
uint32_t HAL_GetTick(void);
void HAL_IncTick(void);
void HAL_Delay(uint32_t Delay);
```

HAL_GetTick() 返回32位HAL节拍计数。默认每1 ms增加一次，因此返回值通常可按毫秒理解。

```
if ((HAL_GetTick() - last_tick) >= timeout_ms)
{
    /* 超时处理 */
}
```

采用无符号减法计算时间差，可以正确处理计数器回绕。1 ms节拍下，32位计数约49.7天回绕一次。不能使用 HAL_GetTick() >= last_tick + timeout_ms，后者在加法溢出时容易误判。

HAL_IncTick() 没有参数和返回值，只负责增加HAL节拍计数。

HAL_Delay() 的 Delay 参数表示等待的毫秒数，没有返回值。函数通过反复读取 HAL_GetTick() 等待时间到达，属于忙等待。它适合操作系统启动前的简单硬件验证，不适合代替 osTimeDly() 写在任务循环中，否则当前任务会持续占用CPU。也不应在中断中调用，因为当前中断可能阻止HAL节拍继续更新，造成无法退出。

注意事项：

- HAL_GetTick() 返回上电后的HAL计数，不表示设备与主站已经连接
- 下载程序后即使IAR没有运行，STM32仍会执行Flash中的程序，HAL计数也会继续增加
- 调试断点可能暂停CPU，节拍中断随之停止，观察到的时间不一定等于真实经过时间
- HAL_GetTick() 分辨率为1 ms，不能精确测量Modbus RTU的字符间隔

3. UART初始化结构体

UART_HandleTypeDef 是串口句柄，既保存初始化配置，也保存收发过程中的状态。

```
UART_HandleTypeDef huart1;
```

成员	作用
Instance	串口寄存器基地址，本为 USART1
Init	UART_InitTypeDef 初始化参数
pTxBuffPtr、TxXferSize、TxXferCount	发送缓冲区、总长度和剩余长度
pRxBuffPtr、RxxferSize、RxxferCount	接收缓冲区、总长度和剩余长度
gState	初始化和发送状态，例如 READY、BUSY_TX
RxState	接收状态，例如 READY、BUSY_RX
ErrorCode	溢出、噪声、帧错误和校验错误等状态位

UART_HandleTypeDef 中的状态和计数由HAL维护。应用程序可以读取状态用于故障判断，但不应直接修改 pRxBuffPtr、RxxferCount、gState 等内部成员。

UART_InitTypeDef 保存通信参数。

成员	本配置	作用
BaudRate	19200	每秒传输的符号数，双方必须一致
WordLength	UART_WORDLENGTH_8B	串口字长
StopBits	UART_STOPBITS_1	1个停止位
Parity	UART_PARITY_NONE	无校验
Mode	UART_MODE_TX_RX	同时允许发送和接收
HwFlowCtl	UART_HWCONTROL_NONE	不使用RTS、CTS硬件流控
OverSampling	UART_OVERSAMPLING_16	16倍过采样，提高常规波特率下的采样稳定性

即使UART配置无校验，Modbus RTU报文末尾仍必须携带CRC16。

4. HAL_UART_Init()

```
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart);
```

huart 指向已经填写 Instance 和 Init 成员的串口句柄。函数检查参数，调用 HAL_UART_MspInit() 配置底层时钟、GPIO和中断，再把波特率、数据位、停止位、校验和收发模式写入USART寄存器。成功返回 HAL_OK，配置错误返回 HAL_ERROR。

```

huart1.Instance = USART1;
huart1.Init.BaudRate = 19200;
huart1.Init.WordLength = UART_WORDLENGTH_8B;
huart1.Init.StopBits = UART_STOPBITS_1;
huart1.Init.Parity = UART_PARITY_NONE;
huart1.Init.Mode = UART_MODE_TX_RX;
huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
huart1.Init.OverSampling = UART_OVERSAMPLING_16;

if (HAL_UART_Init(&huart1) != HAL_OK)
{
    Error_Handler();
}

```

句柄必须在函数调用期间有效，并在后续所有收发操作中使用同一个对象。不能只复制部分句柄成员生成另一个句柄，否则HAL状态与硬件状态会失去对应关系。

5. HAL_UART_Transmit()

```

HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
                                     const uint8_t *pData,
                                     uint16_t Size,
                                     uint32_t Timeout);

```

参数	含义
huart	UART句柄，本传Mbuart，实际指向huart1
pData	待发送缓冲区首地址，发送结束前必须保持有效
Size	发送的数据元素数量；在8位无校验模式下等于字节数
Timeout	整次阻塞发送允许的最长时间，单位为毫秒

函数轮询发送数据寄存器空标志，将缓冲区内容逐字节写入串口，最后等待发送完成标志。发送期间当前任务停留在该函数内，因此它属于阻塞式发送。

返回值	含义
HAL_OK	全部数据已经发送完成
HAL_ERROR	缓冲区为空或长度为0
HAL_BUSY	UART正在执行其他发送操作
HAL_TIMEOUT	在Timeout内未完成发送

报文较短且波特率为19200时，阻塞时间有限；若发送数据量增大或存在更严格的实时任务，应考虑中断或DMA发送。

注意事项：

- Timeout 控制整次发送，不是每个字节各自拥有一份完整超时
- 不能在高优先级中断中调用阻塞式发送
- 发送期间不能修改 pData 指向的缓冲区
- 返回 HAL_OK 后才能统计为成功应答

6. HAL_UART_Receive_IT()

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart,
                                       uint8_t *pData,
                                       uint16_t Size);
```

参数	含义
huart	UART句柄
pData	接收缓冲区首地址，接收完成前必须保持有效且可写
Size	本次准备接收的数据元素数量；8位无校验模式下等于字节数

该函数只登记接收缓冲区和长度，设置 RxState，打开接收及错误中断，然后立即返回。它不会等待数据到达。

返回值	含义
HAL_OK	接收中断已成功启动
HAL_ERROR	缓冲区为空或长度为0
HAL_BUSY	上一次中断接收尚未完成，RxState 不是 READY

每次只接收1字节。

```
static uint8_t MbReceiveByte;

HAL_UART_Receive_IT(MbUart, &MbReceiveByte, 1u);
```

当1字节收到后，本次接收任务结束，HAL调用 HAL_UART_RxCpltCallback()。要继续接收后续字节，必须在回调中再次调用 HAL_UART_Receive_IT()。MbReceiveByte 为静态变量，整个程序运行期间始终有效；不能把即将退出函数的局部变量地址交给中断接收。

7. UART中断和回调

```
void HAL_UART_IRQHandler(UART_HandleTypeDef *huart);
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);
```

HAL_UART_IRQHandler() 的 huart 参数用于确定发生中断的UART及其接收状态。函数读取USART状态标志，处理接收数据、发送状态和错误。它没有返回值，由实际的USART中断入口调用。

```
void USART1_IRQHandler(void)
{
    HAL_UART_IRQHandler(&huart1);
}
```

`HAL_UART_RxCpltCallback()` 是接收完成回调。参数 `huart` 指向完成接收的UART句柄，没有返回值。先比较 `uartHandle->Instance`，确认中断来自USART1，再把收到的字节交给RTU组帧模块，随后重启TIM2一次定时器，最后重新启动下一字节接收。

`HAL_UART_ErrorCallback()` 是错误回调。参数含义与接收完成回调相同。在该函数中增加错误计数、清除溢出标志，并在接收状态恢复为 `READY` 后重新启动接收。

两个回调在HAL库中以弱函数形式提供，应用文件定义同名函数即可覆盖默认空实现。回调运行在中断环境，应保持短小，不能解析完整Modbus报文、调用阻塞发送或执行长时间循环。

USART1接收路径如下。

```
MbPortInit()
-> HAL_UART_Receive_IT() 登记1字节接收
-> USART1收到字节并触发中断
-> USART1_IRQHandler()
-> HAL_UART_IRQHandler()
-> HAL_UART_RxCpltCallback()
-> 字节写入RTU组帧状态
-> 重启TIM2一次定时器
-> 再次调用HAL_UART_Receive_IT()
```

若回调末尾没有重新启动接收，串口通常只能收到第一次申请的1字节。若重复启动时返回 `HAL_BUSY`，说明前一次接收状态尚未结束，不能直接改写HAL内部状态强行恢复。

8. GPIO结构体和函数

```
typedef struct
{
    uint32_t Pin;
    uint32_t Mode;
    uint32_t Pull;
    uint32_t Speed;
    uint32_t Alternate;
} GPIO_InitTypeDef;
```

成员	作用
<code>Pin</code>	一个或多个引脚掩码，例如 <code>GPIO_PIN_6</code>
<code>Mode</code>	输入、输出、模拟或复用模式
<code>Pull</code>	无上下拉、上拉或下拉
<code>Speed</code>	输出边沿速度
<code>Alternate</code>	复用模式下选择连接的片上外设，普通GPIO模式不使用

```

void HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init);
void HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx, uint32_t GPIO_Pin);
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx,
                        uint16_t GPIO_Pin,
                        GPIO_PinState PinState);
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);

```

GPIOx 为端口地址，例如 GPIOF 或 GPIOA。GPIO_Init 指向初始化结构体。GPIO_Pin 为引脚掩码，可以按位或组合多个引脚。PinState 取 GPIO_PIN_SET 或 GPIO_PIN_RESET，分别将输出置1或置0。

HAL_GPIO_Init()、HAL_GPIO_DeInit()、HAL_GPIO_WritePin() 和 HAL_GPIO_TogglePin() 均无返回值。HAL_GPIO_ReadPin() 返回指定引脚当前输入电平。HAL_GPIO_TogglePin() 翻转输出状态，适合基础点灯验证；多个任务同时控制同一引脚时不能依靠翻转表达确定状态。

HAL_GPIO_Init() 调用前必须使用 __HAL_RCC_GPIOx_CLK_ENABLE() 打开对应GPIO端口时钟。

HAL_GPIO_DeInit() 把指定引脚恢复为复位状态。HAL_GPIO_WritePin() 通过置位/复位寄存器修改输出，适合任务和中断使用，但写入电平是否点亮负载取决于硬件有效电平。本Q0低电平有效，因此 GPIO_PIN_RESET 表示点亮。

9. 时钟结构体和函数

RCC_OscInitTypeDef 配置振荡器和PLL。

成员	作用
OscillatorType	指定需要配置HSE、HSI、LSE或LSI中的哪一项
HSEState、HSIState	设置外部或内部高速时钟状态
LSEState、LSIState	设置低速时钟状态
PLL	RCC_PLLInitTypeDef 类型的PLL配置

RCC_PLLInitTypeDef 中的 PLLState 控制PLL开关，PLLSource 选择输入时钟，PLLM 完成输入分频，PLLN 完成倍频，PLLQ 生成系统时钟，PLLQ 供USB、SDIO和RNG等外设使用。

RCC_ClkInitTypeDef 配置系统总线时钟。

成员	作用
ClockType	指定本次要配置的SYSCLK、HCLK、PCLK1和PCLK2
SYSCLKSource	选择系统时钟来源
AHBCLKDivider	SYSCLK到HCLK的分频
APB1CLKDivider	HCLK到PCLK1的分频
APB2CLKDivider	HCLK到PCLK2的分频

```

HAL_StatusTypeDef HAL_RCC_OscConfig(
    const RCC_OscInitTypeDef *RCC_OscInitStruct);

HAL_StatusTypeDef HAL_RCC_ClockConfig(
    const RCC_ClkInitTypeDef *RCC_ClkInitStruct,
    uint32_t FLatency);

```

`HAL_RCC_OscConfig()` 接收振荡器配置结构体，启动HSE并配置PLL。`HAL_RCC_ClockConfig()` 接收总线时钟配置，`FLatency` 为Flash等待周期，本168 MHz对应 `FLASH_LATENCY_5`。两者成功返回 `HAL_OK`，时钟未稳定、参数错误或切换超时返回 `HAL_ERROR` 或 `HAL_TIMEOUT`，因此初始化代码统一检查返回值并进入 `Error_Handler()`。

系统时钟配置错误会同时影响CPU执行速度、UART波特率、SysTick周期和DWT计数换算。修改PLL参数后必须重新检查 `SystemCoreClock` 和各总线频率。

10. NVIC接口

`HAL_UART_MspInit()` 是 `HAL_UART_Init()` 调用的底层初始化回调。参数 `huart` 用于判断正在初始化的串口。USART1分支完成以下操作。

常用NVIC接口如下。

```

void HAL_NVIC_SetPriority(IRQn_Type IRQn,
    uint32_t PreemptPriority,
    uint32_t SubPriority);

void HAL_NVIC_EnableIRQ(IRQn_Type IRQn);
void HAL_NVIC_DisableIRQ(IRQn_Type IRQn);

```

`IRQn` 为中断号，本使用 `USART1_IRQn`。`PreemptPriority` 为抢占优先级，数值越小优先级越高。`SubPriority` 在抢占优先级相同时决定响应顺序，其有效位数受优先级分组影响。`EnableIRQ` 和 `DisableIRQ` 分别允许和禁止该中断，三个函数均无返回值。

只调用 `HAL_UART_Receive_IT()` 不能代替NVIC配置。如果USART1外设中断没有在NVIC中使能，接收申请虽然可能返回 `HAL_OK`，CPU仍不会进入 `USART1_IRQHandler()`。

11. HAL接口使用顺序

```

HAL_Init()
-> 建立HAL时基和基础中断配置
-> HAL_RCC_OscConfig()配置HSE和PLL
-> HAL_RCC_ClockConfig()切换系统及总线时钟
-> HAL_GPIO_Init()配置普通GPIO
-> HAL_UART_Init()配置USART1及其GPIO、NVIC
-> HAL_UART_Receive_IT()启动第一次单字节接收
-> 中断和回调持续接收后续字节
-> HAL_UART_Transmit()在任务中发送Modbus应答

```

注意事项：

- HAL句柄必须与实际外设一一对应，并在全部异步操作期间保持有效

- 中断接收缓冲区必须是全局变量、静态变量或其他长期有效内存
- 所有返回 HAL_StatusTypeDef 的关键调用都应检查结果
- 中断回调只保存数据和恢复接收，协议解析放在任务中完成
- HAL_GetTick() 适合毫秒级超时，RTU字符间隔应使用更高分辨率的计时来源
- CubeMX重新生成代码后，应检查用户回调、中断入口和时基合并代码是否仍然保留

四、数据与地址分配

1. 线圈区

地址	权限	用途
0~15	读写	HMI线圈显示和批量写入测试
16~9999	读写	预留线圈区

线圈只保存 0 或 1。应答时按位打包，第一个线圈放在第一个数据字节的bit0。

2. 保持寄存器区

地址	权限	用途
0	读写	运行时间，由监控任务周期更新
1	读写	最近有效请求状态，由监控任务周期更新
2	读写	有效请求数量，由监控任务周期更新
3	读写	成功发送应答数量，由监控任务周期更新
4	读写	CRC错误数量，由监控任务周期更新
5	读写	接收溢出数量，由监控任务周期更新
6~99	读写	普通寄存器区
100~109	读写	HMI批量读写测试区
110~9999	读写	扩展寄存器区

保持寄存器为16位，地址 0~9999 均允许读写。地址 0~5 虽然可以由主站写入，但监控任务会周期更新这些状态值，因此写入值不会长期保持。

3. HMI变量绑定

HMI区域	从站地址	操作
在线与诊断	保持寄存器 0~5	周期读取
线圈测试	线圈 0~15	0x01 读取， 0x0F 批量写
寄存器测试	保持寄存器 100~109	0x03 读取， 0x10 批量写

HMI内部变量不等于Modbus从站地址。画面地址还可能存在从0开始和从1开始的显示差异。联调时先读保持寄存器 100，确认地址对应关系后再批量配置。

4.功能模块

模块	主要输入	主要输出	职责
uC/OS-II任务	系统节拍和任务状态	任务调度	组织通信、监控和状态任务
串口通信	请求字节	完整RTU帧	接收、分帧和发送
Modbus从站	完整请求和数据区	正常应答、异常应答或静默	协议检查和功能码处理
HMI	用户操作和从站应答	主站请求和画面状态	轮询、写入、超时和显示

模块关系如下。

HMI发送请求

-> USART1逐字节接收

-> RTU静默时间形成完整帧

-> Modbus从站检查CRC、站号和功能码

-> 数据区被读取或写入

-> 从站生成应答

-> HMI检查应答并刷新画面

六、程序启动流程

程序入口在 app/main.c。

顺序	函数	作用
1	HAL_Init()	初始化HAL库和基础时基
2	SystemClock_Config()	配置系统时钟为168 MHz
3	MX_GPIO_Init()	初始化GPIO
4	MX_USART1_UART_Init()	初始化USART1
5	MbDataReset()	清空线圈和保持寄存器数据区
6	MbSlaveInit(1)	设置Modbus从站地址
7	MbPortInit(&huart1)	初始化串口接收状态和RTU组帧模块
8	OSInit()	初始化uC/OS-II内核
9	AppTasksCreate()	创建TakeFramesem、LED更新信号量和应用任务
10	OSStart()	启动调度器

OSStart() 之后，主函数不再按普通顺序继续执行。CPU由uC/OS-II调度到各个任务中运行。

三个应用任务分工如下。

任务	周期动作	目的
AppModbusTask	周期调用 MbPortReceiveFrame()	处理完整Modbus请求
AppLedTask	等待信号量后刷新状态灯	显示主站心跳在线状态
AppMonitorTask	周期更新状态寄存器	提供运行时间和通信统计

本使用 AppLedUpdateSemaphore 完成任务同步。信号量初值为0，表示没有LED更新事件。通信任务收到合法心跳后发布信号量，监控任务完成3秒心跳超时检查后也发布信号量；LED任务永久等待信号量，收到通知后读取 MB_PORT_STATISTICS.online 并更新Q0。

使用信号量的原因是Q0不需要固定周期反复检查状态，只需要在收到合法心跳或完成超时检查后刷新。信号量使LED任务在无事件时进入等待态，减少无效轮询，同时把“在线状态判定”和“Q0显示”两个任务连接起来。

```
OSSemCreate(0)
-> AppLedTask调用OSSemPend等待
-> 通信任务或监控任务调用OSSemPost
-> LED任务由等待态变为就绪态
-> 读取online并刷新Q0
-> 再次等待信号量
```

信号量只表示“在线状态可能变化，需要刷新Q0”，不保存在线值。串口只有Modbus任务访问，因此没有使用该信号量保护串口，也没有为了信号量增加多个任务争用USART1。

处理位置	信号量操作	处理结果
AppTasksCreate()	OSSemCreate(0u)	创建初值为0的事件信号量 TakeFramesem，失败则停止启动
AppLedTask()	OSSemPend(..., 0u, &error)	无事件时永久等待，获得通知后读取 online 并刷新Q0
AppModbusTask()	OSSemPend(TakeFramesem, 0u, &error) 后处理完整帧	等待TIM2封帧通知
AppMonitorTask()	心跳超时检查后 OSemPost()	通知Q0显示当前检查结果

创建、等待和发布都检查错误。信号量对象使用 static 限制在任务模块内部。先更新 online 再发布信号量，保证Q0任务醒来后读取的是最新状态。重复通知只会让Q0再次读取最新值，不改变Modbus数据，也不会把通知次数当成在线状态。

七、RTU接收与分帧

从串口接收缓存取数据时必须关中断，因为底层串口的中断服务程序随时可能向该缓存写入新数据，如果不关中断，主程序在取数据过程中一旦被中断打断，缓存内容被修改，就会导致取出的数据错乱、不完整或指针异常，这种现象称为竞态条件；正确做法是：关中断之后 快速取出全部数据需要几微秒内完成，之后立即开中断，这样能保证数据读取的原子性和一致性，同时将关中断时间压到最短，避免影响系统实时性。

USART1采用中断方式逐字节接收。每收到1字节，回调函数只把字节放入RTU组帧模块，然后重新启动TIM2一次定时器。最后一个字节后静默达到 `t3.5`，由TIM2中断确认一帧接收完成。复制接收缓冲区时短暂进入临界区，防止中断同时写入新数据。

中断回调只处理短动作，不解析协议，不发送应答。协议解析放在任务中执行，避免中断占用时间过长。

收到字节

-> 保存到接收缓冲区

-> 重启TIM2一次定时器

-> 等待下一字节

-> 静默达到t3.5

-> TIM2中断确认完整帧

-> Post TakeFramesem

-> Modbus任务取出并处理请求

Modbus任务由 `TakeFramesem` 唤醒，不再周期轮询。

接收溢出时整帧丢弃并计数。CRC错误、站号不匹配和广播读请求均不返回应答。

八、从站协议处理

`MbSlaveProcess()` 是从站协议入口。输入为完整请求帧，输出为应答帧长度。

公共检查顺序如下。

检查长度

-> 检查CRC

-> 检查从站地址

-> 记录有效请求

-> 按功能码分派

情况	处理
长度过短	静默
CRC错误	静默并增加CRC错误计数
站号不匹配	静默
写请求的Byte Count与实际Data长度不一致，CRC正确	异常 03
不支持的功能码	异常 01
起始地址或连续地址越界	异常 02
数量、长度或字节数错误	异常 03

异常应答格式为：

从站地址 + 原功能码|0x80 + 异常码 + CRC低字节 + CRC高字节

例如 0x10 写寄存器发生地址错误时，异常功能码为 0x90。

九、四个功能码

1. 0x01 读取线圈

主站给出起始地址和数量，从站返回连续线圈值。线圈按位打包，第一个线圈放在第一个数据字节的bit0。

请求格式：

地址 01 起始地址 起始地址 数量 数量 CRC

应答格式：

地址 01 字节数 线圈数据 CRC

检查内容包括请求长度、数量范围、地址范围和应答缓冲区容量。数量范围为 1~2000。

2. 0x03 读取保持寄存器

主站给出起始地址和数量，从站返回连续保持寄存器。每个寄存器占2字节，高字节在前。

请求格式：

地址 03 起始地址 起始地址 数量 数量 CRC

应答格式：

地址 03 字节数 寄存器数据 CRC

数量范围为 1~125。应答字节数等于寄存器数量乘2。状态寄存器 0~5 允许读取。

3. 0x0F 写多个线圈

主站将多个线圈状态按位打包后发送给从站。从站检查全部参数合法后再逐位写入。

请求格式：

地址 0F 起始地址 起始地址 数量 数量 字节数 线圈数据 CRC

成功应答：

地址 0F 起始地址 起始地址 数量 数量 CRC

数量范围为 1~1968。请求中的Byte Count应等于 $(\text{数量} + 7) / 8$ ，实际线圈Data长度还必须等于Byte Count。两层关系任意一项错误且CRC正确时返回异常 03。

4. 0x10写多个保持寄存器

主站将多个16位保持寄存器写入从站。每个寄存器高字节在前。

请求格式：

```
地址 10 起始地址Hi 起始地址Lo 数量Hi 数量Lo 字节数 寄存器数据 CRC
```

成功应答：

```
地址 10 起始地址Hi 起始地址Lo 数量Hi 数量Lo CRC
```

数量范围为 1~123。Byte Count应等于 数量*2，实际寄存器Data长度还必须等于Byte Count。保持寄存器地址0到9999都允许写入，地址0到5会被监控任务周期覆盖。写入完成后使用 0x03 回读，可以确认地址、数量和字节序是否正确。

十、HMI通信

HMI作为主站，负责构建请求、发送请求、等待应答和更新画面变量。STM32从站不会主动上报数据。

1. TouchWin函数功能块

TouchWin函数功能块采用C语言的标识符、变量、数组、判断和循环语法。它由TouchWin编译，下载到触摸屏后在HMI内部运行，并通过TouchWin提供的内部寄存器和通信函数访问画面及串口。

函数功能块要求TouchWin V2.C.6或更高版本，并且必须下载到真实HMI后运行，不支持在线模拟和离线模拟。

2. 公共函数和功能函数

TouchWin把函数分为公共函数和功能函数。

类型	写法	调用方式
公共函数	有正常函数头，可有参数和返回值	由其他函数调用
功能函数	无参数、无返回值，不写函数头，只写函数体	由功能键或功能域直接调用

例如业务函数块中只有下面的声明，而没有 void HmiReadCoils16(void) 这样的函数头。

```
UINT ModbusCrc16(BYTE *data, WORD length);

BYTE sendBuffer[8];
WORD startAddress;
```

第一行是公共函数原型，后面的内容已经属于功能函数的函数体。功能函数由TouchWin中的函数名称标识，功能键选择“函数调用”后直接选择该名称。

公共函数可以被多个功能函数重复调用。CRC计算和在线状态更新属于通用逻辑，所以单独写成公共函数；四个功能码的画面输入、报文格式和应答解析不同，因此分别写成功能函数。

3. TouchWin预定义数据类型

TouchWin预先定义了适合画面和通信使用的数据类型。

类型	TouchWin定义
BOOL	unsigned char
BYTE	unsigned char
WORD	unsigned short
UINT	unsigned int
DWORD	unsigned long

TRUE 为1， FALSE 为0。报文必须使用 BYTE 数组，因为Modbus RTU按字节发送；地址、数量和寄存器值使用 WORD，在放入报文时再拆成高、低两个字节。

4. HMI内部对象

TouchWin函数功能块中的 PSW 和 PSB 不是普通局部数组，而是HMI内部对象。画面组件和函数功能块可以通过这些地址交换数据。

对象	含义	函数中的访问方式
PSW	HMI内部16位寄存器	PSW[300] 直接读写
PSB	HMI内部位对象	GetPSBStatus()、SetPSB()、ResetPSB()
PFW	HMI掉电保持寄存器	通过 Read()、Write() 等函数访问
PLC	TouchWin预定义通信口编号	传给通信函数

例如起始地址输入框绑定 PSW300。用户在画面输入地址后，函数通过 startAddress = PSW[300] 取得该值。读取成功后，函数把线圈值写到 PSW302，画面的数据显示组件绑定 PSW302 即可显示结果。

PSW 中每个单元都是一个 WORD。自定义报文表格虽然每格绑定一个PSW，但每个报文字节只能是 0~255，所以发送前还要检查PSW数值并使用 LOBYTE() 取出低8位。

5. 宏和通信函数

HIBYTE() 和 LOBYTE()

HIBYTE(value) 取得16位数据的高字节，LOBYTE(value) 取得低字节。

```
sendBuffer[2] = HIBYTE(startAddress);
sendBuffer[3] = LOBYTE(startAddress);
```

如果 startAddress 为 0x1234，则放入报文的两个字节依次为 0x12 和 0x34。Modbus的地址、数量和寄存器值均采用高字节在前；CRC发送顺序相反，先放CRC低字节，再放CRC高字节。

Enter() 和 Leave()

```
void Enter(BYTE comID);
void Leave(BYTE comID);
```

Enter(PLC) 取得PLC通信口资源， Leave(PLC) 释放资源。两者通过信号量保证一次发送和接收过程按同步方式完成，防止另一个函数同时占用同一串口。它们必须成对使用，并且 Leave() 应放在发送、接收和判断全部结束之后。

Send()

```
BOOL Send(BYTE comID, BYTE *sendBuffer, WORD length);
```

参数	含义
comID	通信口，本使用 PLC
sendBuffer	发送字节数组的起始地址
length	实际发送字节数

返回 TRUE 表示TouchWin接受了本次发送操作，返回 FALSE 表示发送失败。它不能证明从站已经正确处理请求，因此发送成功后仍必须调用 Receive() 等待应答。

Receive()

```
WORD Receive(BYTE comID, BYTE *receiveBuffer, WORD length,
              WORD totalTimeOut, BYTE byteTimeOut);
```

参数	本取值	含义
comID	PLC	PLC通信口
receiveBuffer	接收数组	保存收到的应答字节
length	按功能码计算	希望接收的应答长度
totalTimeOut	500	总接收超时500 ms
byteTimeOut	6	字节间接收超时，按手册建议取6

返回值是实际收到的字节数。返回0通常表示在超时时间内没有收到数据；返回长度与正常应答长度不一致时，还要判断它是否为固定5字节的Modbus异常应答。

SetPSB() 和 ResetPSB()

SetPSB(256) 把主站在线位设置为1， ResetPSB(256) 把它清零。PSB不能像PSW一样在函数中直接使用数组下标赋值。

一次业务事务：

用户在画面输入参数

- > 参数保存到PSW
- > 按下功能键调用对应功能函数
- > 功能函数读取PSW并检查范围
- > 把站号、功能码、地址和数量写入sendBuffer
- > 公共函数ModbusCrc16计算CRC
- > Enter取得PLC通信口
- > Send发送请求
- > Receive等待应答
- > 检查实际长度、CRC、站号、功能码和回显字段
- > 把数据、结果码和异常码写入PSW
- > Leave释放PLC通信口
- > 画面组件显示新的PSW值

6. 四个功能块

四个功能块对应四个功能码。

功能块	功能码	作用
HmiReadCoils16	0x01	读取1到16个线圈
HmiWriteCoils16	0x0F	写入1到16个线圈
HmiReadHolding4	0x03	读取1到4个保持寄存器
HmiWriteHolding4	0x10	写入1到4个保持寄存器

各函数块的画面接口：

功能块	输入PSW	数据输出PSW	状态输出PSW
HmiReadCoils16	300 起始地址, 301 数量	302 线圈位组合值	303 结果, 304 长度, 305 异常码, 306 参数错误
HmiWriteCoils16	310 起始地址, 311 数量, 312~313 线圈数据	无, 写完用 0x01回读	314 结果, 315 长度, 316 异常码, 317 参数错误
HmiReadHolding4	320 起始地址, 321 数量	322~325 寄存器值	326 结果, 327 长度, 328 异常码, 329 参数错误
HmiWriteHolding4	330 起始地址, 331 数量, 332~335 寄存器值	无, 写完用 0x03回读	336 结果, 337 长度, 338 异常码, 339 参数错误

读取函数

HmiReadCoils16 和 HmiReadHolding4 的请求固定为8字节。两者先根据数量计算正常应答长度，再调用 Receive()。如果实际收到5字节，则按异常应答检查；否则按正常应答检查。

0x01 需要把返回的数据字节组合成线圈位。第一个数据字节保存前8个线圈，第二个数据字节保存后8个线圈，组合后写入 PSW302。0x03 每个寄存器占两个字节，函数按高字节在前组合成WORD，然后依次写入 PSW322~PSW325。

写入函数

HmiWriteCoils16 根据数量计算Byte Count，把 PSW312 和 PSW313 中的低、高8位线圈数据放入请求。最后一个数据字节中不属于本次数量的多余位会被清零。正常应答必须回显原请求的起始地址和数量。

HmiWriteHolding4 把 PSW332~PSW335 中的每个WORD拆成高、低两个字节，Byte Count等于数量乘2。正常应答同样检查起始地址和数量回显。写功能块成功只表示从站接受写请求，验收时还要调用相应读取功能块回读。

7. 结果码

四个函数使用统一结果码。

结果值	含义
0	正在执行或尚无结果
1	正常应答
2	发送失败
3	接收长度错误或超时
4	CRC错误
5	站号、功能码或回显内容不匹配
6	HMI输入参数越界
7	收到Modbus异常应答

结果值 6 表示HMI本地参数检查失败，通常没有发出Modbus请求。结果值 7 表示从站收到请求并返回了合法异常帧，应继续查看异常码。

判断应答时不能只看 Receive() 是否返回数据。正常应答还必须同时满足CRC正确、站号为1、功能码一致、字节数正确；

0x0F 和 0x10 还要检查起始地址与数量回显。任一字段不匹配，结果置为5。

8. 自定义报文函数块

自定义报文画面使用 PSW360~PSW375 保存最多16个发送字节，使用 PSW380~PSW395 显示最多16个接收字节。

函数	作用
HmiCustomFrameCalculateCrc	根据输入长度计算CRC并写入 PSW355
HmiCustomFrameSend	自动或手动处理CRC，发送报文并解析应答
HmiCustomFrameClear	清空输入参数、发送表格和接收表格

PSW350 表示发送报文总长度，包含最后两个CRC字节。PSW357=0 时使用自动CRC，函数会覆盖输入表格最后两格；PSW357=1 时保留用户输入的CRC，可用于测试从站对CRC错误帧的静默处理。

自定义发送仍然使用 Enter()、Send()、Receive() 和 Leave()，但会根据请求功能码和数量估算正常应答长度。收到应答后，函数把每个字节写入接收表格，并检查CRC、站号、功能码、字节数或写应答回显。错误CRC请求通常得不到从站应答，因此HMI显示的是等待应答超时，而不是“接收CRC错误”。

十、在线状态与运行时间

在线状态不能只看某个数值是否在变化。主站在线应以事务结果为依据。HMI连续收到正确应答时显示在线，连续多次失败后显示离线。离线后仍应继续周期请求，后续通信恢复时才能重新进入在线状态。

保持寄存器 0 保存运行时间。该值来自STM32运行后的毫秒计数换算，程序下载到Flash后，只要板子持续供电且没有被调试器暂停，运行时间就会继续增加。IAR没有打开并不影响芯片独立运行。

保持寄存器 0~5 含义如下。

地址	含义
0	运行时间
1	最近有效请求状态
2	有效请求数量
3	成功发送应答数量
4	CRC错误数量
5	接收溢出数量

十一、注意事项

层级	观察量	正常现象
硬件	电源、SWD、GPIO	可以下载，GPIO可控制
时基	HAL时间、OS时间	持续增加
任务	任务计数	按周期增加
串口	接收字节数	主站请求时增加
分帧	完整帧数	每条请求形成一帧
协议	有效请求数、CRC错误数	正确请求增加，正常通信CRC错误稳定

层级	观察量	正常现象
应答	发送应答数	非广播正常事务增加
HMI	结果码、实际长度、异常码	正常为成功，错误能定位到对应类型

常见问题：

现象	优先检查
IAR无法连接	ST-Link、SWD配置、供电、调试器类型
HMI一直超时	串口参数、A/B接线、从站地址、从站是否运行
CRC错误增加	波特率、校验位、字节顺序、CRC算法
返回异常 02	起始地址和连续数量是否越界
返回异常 03	数量范围、字节数、请求长度
写入后回读不一致	地址偏移、字节序、HMI绑定对象